

TaskManagerPro - Ejercicio Técnico

El Problema

Muchas personas manejan proyectos complejos: desarrollo de software, campañas de marketing, construcción de casas, lanzamiento de productos. Todos estos proyectos comparten algo: tienen **tareas grandes, subtareas que desglosan el trabajo, y hitos importantes que marcan progreso**.

El problema hoy es que no existe una herramienta simple que permita a usuarios:

1. Organizar trabajo en múltiples niveles:

- Una tarea principal ("Desarrollar API REST")
- Subtareas que desglosan el trabajo ("Implementar endpoints", "Autenticación", "Tests")
- Hitos que marcan objetivos importantes ("Primer MVP listo el 20 de mayo", "Code review completado")

2. Ver el panorama completo:

- Dónde están en el proyecto (qué tareas están en progreso)
- Qué subtareas quedan por hacer
- Cuáles son los hitos críticos que no pueden fallar

3. Compartir información con otros sistemas:

- Un usuario quiere que sus hitos aparezcan en Google Calendar
- Otro necesita exportar los hitos a XML para pasarlos a su jefe
- Otro quiere tenerlos en JSON para integrar con otra aplicación

Actualmente, usuarios tienen que usar múltiples herramientas: una para tareas, otra para calendario, otra para reportes. **La solución debe ser una única aplicación que maneje todo esto de forma integrada y flexible.**

Requisitos Funcionales

Gestión de Tareas

- Crear, leer, actualizar y eliminar tareas
- Cada tarea tiene: título, descripción, fecha de inicio y fin, prioridad, estado (No iniciada, En progreso, Completada, Retrasada)
- Cada tarea puede tener múltiples subtareas

Subtareas

- Crear, leer, actualizar y eliminar subtareas dentro de una tarea
- Cada subtask tiene: descripción, estado (Pendiente, Completada), fecha de vencimiento opcional, notas adicionales
- Las subtareas ayudan a desglosar el trabajo grande en pasos manejables

Hitos (Milestones)

- Crear, leer, actualizar y eliminar hitos dentro de una tarea
- Cada hito tiene: título, descripción, fecha objetivo, estado (Pendiente, Completado, Retrasado)
- Los hitos representan objetivos importantes (entregas, reviews, validaciones) que marcan progreso real
- Un hito es diferente de una subtask: una subtask es "trabajo que debo hacer", un hito es "fecha importante que debo cumplir"

Exportación de Hitos

- Exportar hitos a **JSON** (formato datos)
- Exportar hitos a **XML** (formato estándar)
- Exportar hitos a **iCal** (.ics, para sincronizar con Google Calendar, Outlook, Apple Calendar, etc.)
- Cada exportación debe incluir título, descripción, fecha objetivo y estado del hito

Autenticación y Seguridad

- Registro e inicio de sesión de usuarios
- Cada usuario solo ve sus propias tareas, subtareas y hitos
- Autenticación con JWT

Búsqueda y Organización

- Listar tareas con paginación y filtrado

- Buscar tareas por título o descripción
- Ver todas las subtareas de una tarea
- Ver todos los hitos de una tarea

Notificaciones Asincrónicas (BONUS FEATURE)

- Sistema de notificaciones basado en eventos de tareas
- Las notificaciones se crean de forma **asincrónica** sin bloquear al usuario
- **Eventos que generan notificaciones:**
 - Cuando se crea una tarea: "Nueva Tarea Creada - [título]"
 - Cuando se completa una tarea: "Tarea Completada - [título]"
 - Verificación automática cada hora: "Tarea Vencida - [título]" para tareas con EndDate vencida
- **Endpoints de Notificaciones:**
 - `GET /api/v1/notifications` - Obtener todas las notificaciones del usuario (paginadas)
 - `GET /api/v1/notifications/unread` - Contar notificaciones no leídas
 - `PATCH /api/v1/notifications/{id}/read` - Marcar notificación específica como leída
 - `PATCH /api/v1/notifications/read-all` - Marcar todas las notificaciones como leídas
- **Estructura de Notificación:**
 - `notificationId` - ID único
 - `userId` - Usuario propietario (aislamiento de datos)
 - `title` - Título corto ("Nueva Tarea Creada")
 - `message` - Detalle completo ("Se creó la tarea: Hacer reportes")
 - `type` - Tipo de evento ("TaskCreated", "TaskCompleted", "TaskOverdue")
 - `isRead` - Estado de lectura
 - `createdAt` - Timestamp de creación
- **Implementación técnica:**
 - Usa **Hangfire** para ejecutar jobs en background
 - No bloquea al usuario al crear/completar tareas
 - Reintentos automáticos si un job falla
 - Dashboard de Hangfire para monitoreo en `/hangfire`

Requisitos No Funcionales

- **Backend:** .NET 10 con arquitectura limpia (Domain, Application, Infrastructure, API)
- **Frontend:** Angular moderno con componentes standalone, Angular Material para UI
- **Persistencia:** Base de datos relacional (SQLite para desarrollo)
- **Autenticación:** JWT Bearer Tokens

- **API:** Endpoints RESTful versionados
 - **Código:** Limpio, testeable, siguiendo SOLID y Clean Architecture
 - **UI:** Profesional, responsive, intuitiva
-

Consideraciones Técnicas

Backend

Debes pensar en:

- ¿Cómo modelar la relación entre Task, SubTask y Milestone?
- ¿Cómo hacer que cada usuario solo vea sus propios datos?
- ¿Cómo implementar múltiples exportadores sin duplicar código? (Patrón Strategy, Factory)
- ¿Cómo manejar eliminación segura de tareas sin perder el historial?
- ¿Cómo desacoplar la creación de notificaciones del flujo principal de tareas? (Hangfire + Background Jobs)
- ¿Cómo garantizar que un job que falla se reintente sin perder datos?
- ¿Cómo ejecutar verificaciones periódicas (tareas vencidas) sin bloquear la API?

Frontend

Debes pensar en:

- ¿Cómo mostrar tareas, subtareas y hitos en una interfaz clara?
 - ¿Cómo permitir crear/editar subtareas y hitos dentro del mismo formulario de tarea?
 - ¿Cómo indicar visualmente la diferencia entre una subtask y un hito?
 - ¿Cómo hacer que la exportación sea fácil y accesible?
-

Stack Sugerido

Aspecto	Tecnología
Backend	.NET 10, ASP.NET Core, EF Core
Frontend	Angular (standalone), Angular Material, Signals
Base de Datos	SQLite (desarrollo), SQL Server (producción)

Aspecto	Tecnología
Autenticación	JWT Bearer Tokens
Exportación	Librerías estándar (System.Xml.Linq, System.Text.Json, iCal.NET o similar)

Criterios de Éxito

- ✓ Usuario puede crear tareas con subtareas y hitos
 - ✓ La interfaz distingue claramente entre subtareas (trabajo) e hitos (objetivos)
 - ✓ Exportar hitos a JSON, XML e iCal funciona correctamente
 - ✓ Solo el usuario propietario ve sus datos
 - ✓ Paginación y búsqueda funcionan
 - ✓ Código sigue Clean Architecture sin acoplamiento
 - ✓ Sin errores de consola
 - ✓ UI es profesional y accesible
 - ✓ **[BONUS] Notificaciones se crean asincrónicamente sin bloquear al usuario**
 - ✓ **[BONUS] Endpoints de notificaciones funcionan (GET, PATCH)**
 - ✓ **[BONUS] Verificación automática de tareas vencidas cada hora**
 - ✓ **[BONUS] Hangfire dashboard accesible para monitoreo**
-

Repositorio

<https://github.com/nmino1984/TaskManagerPro>

Rama principal: `main`